

技术债务

黄志安教授

邮箱: huang.za@cityu-dg.edu.cn

课程大纲

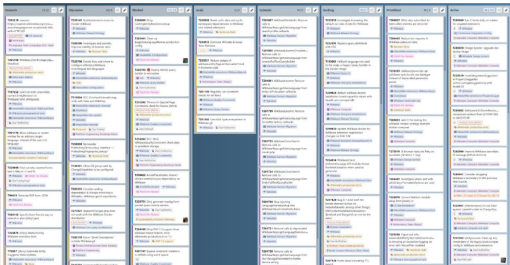
- ◆ 何为技术债务
- ◆ 识别技术债务
- ◆ 管理技术债务
- ◆ 实践中的技术债务管理

技术债务

- ◆ 技术债务 (TD)
 - 这一比喻最初于1992年提出[1]。其核心思想是：采用**"快速粗糙"**的方式解决问题**会形成技术债务** (TD)。
 - 指一种在短期内看似便捷的设计或构建方法，但会形成特定技术环境，**导致相同工作在后期实施的成本高于当前成本** (包括随时间累积的成本增加) [2]。
- ◆ 导致技术债务后果的决策示例
 - "在截止日期前我们没有时间协调这两个数据库，因此先编写粘合代码保持同步，待产品发布后再进行协调。"

示例

任务看板中的技术债务



<https://addshore.com/2021/06/tackling-technical-debt-big-and-small-in-wikidata-and-wikihouse/>

媒体领域技术债务的影响

EE Times

HOME NEWS PERSPECTIVES DESIGNLINES PODCAST EDUCATION STORE

DESIGNLINES | EE LIFE

Engineers Struggle with Wasted Time and Technical Debt

By Cade Atwell 09.06.2021 1

Share Post Share on Facebook Share on Twitter in

How tech debt from legacy solutions will impact business' bottom line

Article by Pure Storage VP & Field CTO Iain Johnson

Public and private-sector organisations are making massive investments in digital transformation to adapt to the new economy. However, many investments rely on building new applications on legacy IT architecture. This technology approach is leading to heavy levels of tech debt, lower organisational performance and increasing cyber risk.

Organisations that take a halfhearted approach to digital transformation may find themselves with tangled webs of tech debt and systems that are business-critical. A recent EY survey found that tech debt currently accounts for 42% of IT business events. 84% of respondents reported increasing spending on tech debt.

Technical debt occurs because of interdependencies between new systems and legacy IT architecture that often require hard coding. This creates IT infrastructure for the organisation, which is often challenging to maintain. Clear warning signs of tech debt include hard coding and financial that necessitate as cloud solutions.

Some tech debt is unavoidable because it is a cost of doing business as part of the transition from old to new technology. However, too much tech debt increases the total cost of ownership of IT for organisations and directly impacts competitiveness because it is prone to security vulnerabilities, operational, and security implications for businesses.

Tech debt increases the total cost of ownership of IT for organisations and directly impacts competitiveness because it is prone to security vulnerabilities, operational, and security implications for businesses.

Cyber criminals may find it much easier to target organisations that have higher levels of tech debt due to the likelihood of greater security gaps and vulnerabilities from disparate systems and a lack of IT integration.

Just like financial debt, organisations cannot afford to sustain tech debt in the long term. It is a critical business issue rather than purely a technical one, such, tech debt must be addressed in terms of its broader organisational impacts over the longer term.

Every workplace has its distractions that can dampen productivity, and the engineering sector is no different, no matter the discipline. Emails, text messages, meetings, and social media also play a significant role in wasted time, which makes it challenging to get to the tasks at hand. Add to that technical debt, and it becomes difficult to stay on track of projects and it generates increased costs. According to a report from Stepsize, engineers spend around six hours per week dealing with technical debt, or the amount of time not working toward critical goals. To put that into another perspective – engineers spend roughly 35% working on maintenance and legacy systems, 50% of which is spent on technical debt. As with money debt, if technical debt isn't paid, it accrues interest, making it tougher to implement changes.

媒体领域技术债务的后果

Forbes

5,132 views | May 30, 2016, 09:33pm

Technical Debt: The Silent Company Killer

Falon Fatemi

Much like a poor diet, technical debt starts with good intentions but ends in frustrating system failures. But unlike unhealthy meals, technical debt is nearly unavoidable: Today's agile developers focus on delivering software quickly to solve a market need. Developers who over-architect or over-code software only delay launch and waste money.

So while shortcuts and assumptions help developers iterate quickly, they also cause software to accrue technical debt. Left unaddressed, this debt bogs down systems, crashes software, and causes multi-month delays in product releases.

媒体技术债务的解决方案

Forbes

measuring

Measuring And Managing Technical Debt

John Kodumal, CTO and cofounder of LaunchDarkly, says, "Technical debt is inevitable in software development, but you can combat it by being proactive: establishing policy, convention, and processes to amortize the cost of reducing debt over time. This is much healthier than stopping other work and trying to dig out from a mountain of debt."

Kodumal recommends several practices, such as "establishing an update policy and cadence for third-party dependencies, using workflows and automation to manage the life cycle of feature flags, and establishing service-level objectives."

To help reduce the likelihood of introducing new technical debt, consider the following best practices:

- Standardize naming conventions, documentation requirements, and reference diagrams.
- Instrument CI/CD and test automation to increase release frequency and improve quality.
- Develop code refactoring as skills and patterns practiced by more developers.
- Define the architecture so developers know where to code data, logic, and experiences.
- Conduct regular code reviews to help developers improve coding practices.

How to minimize new technical debt

With test automation and a commitment to not let technical debt grow, developers can make a solid business case, especially when staffing and money are tight.

Forbes

FORBES > INNOVATION > ENTERPRISE TECH

Here's What You Can Do To Manage Your Technical Debt

Forrester Contributor

Aug 6, 2020, 09:03am EDT

- Demonstrate value.** Those outside of the IT environment may struggle to see the business value of maintaining a technology inventory. There is a cost to managing it, both in the form of people and resources.
- Ensure process compliance.** Teams, seeing this process as a potential delay or blocker to what they want to do, may find ways to evade TLM when the process doesn't produce the result they want, and over time, this dissatisfaction may result in the TLM process being shut down.
- Avoid delays.** There is great business cost in a TLM process that takes months; new technologies may be essential for an organization's survival, and requests for new technology should be evaluated in a timely manner.

InfoWorld

Initiate an inventory of vendors as are nonstandard, and give of vulnerability over TLM requires automation and distributed spreadsheets, Forrester on management database.

7

技术债务

产生债务→支付利息→最终偿还

◆ 典型场景：

冲刺1	冲刺6	冲刺10	
“将模块X作为API暴露给其他模块使用？我们根本不需要它！ 没必要 为此设计。”	“糟了，我们确实需要它！先 绕过 这个问题确保当前版本能发布吧。”	“ 重构 时刻到了——必须修复这个权宜之计！”	注：API开发属于功能开发范畴，而非技术债务。
错误 - 造成损失 技术债务	支付利息 技术债务	偿还 技术债务	

技术债务的“利息支付”示例

技术债务

- ◆ 例如：遗留代码缺乏单元测试导致新开发、测试和调试耗时更长
- ◆ 例如：过度简化的设计难以适应环境变更，看似简单的环境调整却需要大规模代码重构
- ◆ 例如：高缺陷系统导致产品支持成本高昂（非软件利息支付）
- ◆ 例如：脆弱系统导致每次修复漏洞都会引发意外副作用（如新增漏洞），使单次修复演变为多漏洞修复项目
- ◆ 例如：缺陷报告频发，导致生产系统修复缺陷耗费大量时间，致使新功能开发完全停滞
- ◆ 例如，由于开发效率低下导致编辑-编译-调试周期过长

环境（非代码导致的技术债务）

技术债务的反例

- ◆ 许多延迟或未完成的工作并不属于技术债务：
 - 功能待办事项、延后功能、砍掉的功能
- ◆ 通常而言，若“相同（开发）工作在后期实施的成本不高于当前实施成本”，则不构成技术债务。

技术债务实践案例 [8]

- ◆ 恩斯特等人对1831名软件工程师和架构师的调查发现：
 - 31%将技术债务视为待办事项的隐含组成部分
 - 25%明确将技术债务纳入待办事项
- ◆ 技术债务项通常不存在于正式冲刺待办事项中，往往记录在临时管理的 *影子待办事项* 中（即“非正式”记录，如团队或成员的私人笔记、源代码提交注释等）。
- ◆ 冲刺待办事项包含多种不同类型的项（例如新功能、缺陷/错误和改进项）。技术债务通常归类为改进项。
- ◆ 当软件中存在技术债务时，**唯一显著有效的减少方式是重构软件。**

技术债务的影响

- ◆ 在某些大型组织中，用于管理技术债务的开发时间相当可观（平均占整体开发时间的25%，有观点指出高达40%-50%）。

企业视角

- ◆ 许多公司往往以牺牲
 ■ 牺牲长期持续发布新版本的能力——包括高质量、快速反馈和低投入——
以换取
 ■ 短期功能——这些功能虽能快速修复问题，却会在长期拖慢团队步伐。
- ◆ 这种策略使企业更容易受到竞争对手的冲击

同一故事的两面：企业视角[2]

◆ 商业视角

- 缩短上市时间
- 保留初始投资
- 延迟开发支出

◆ 业务人员往往对收益和成本持乐观态度。这些态度通常是无意识的。

◆ 技术视角

```
graph TD; A[承担债务] --> B[债务降低生产力]; B --> C[减少用于开发新功能]; C --> D[降低速度]; D --> A;
```

◆ 技术人员往往对收益和成本持悲观态度。这些态度往往是无意识的。

技术债务及其基本管理

- ◆ 技术债务按维度可分为：
 1. 类型（例如设计、测试或文档债务），
 2. 故意性（故意或无意），
 3. 时间跨度（短期或长期），以及
 4. 关注程度。
- ◆ （流行）技术债务管理理念
 - 识别技术债务并通过软件项目的产品待办事项列表进行追踪。讨论这些债务。
 - 例如，将以下内容放入待办事项列表：
在截止日期前整合这两个数据库，因此将编写临时粘合代码维持同步，
发布后再进行整合。”

运用这些标准评估技术债务

技术维度 债务

维度1： 技术债务的类型

- ◆ 技术债务不仅是编码问题。
- ◆ 重构消除代码异味**无法**解决所有问题。

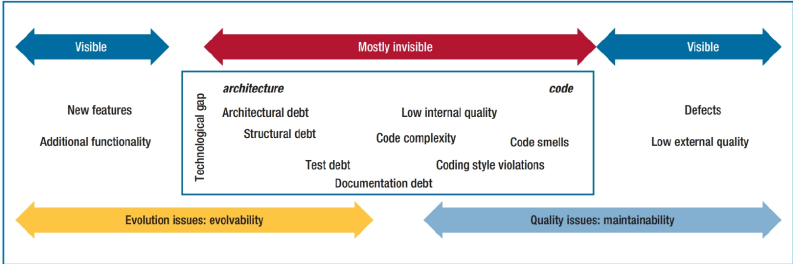


FIGURE 1. The technical debt landscape. On the left, evolution or its challenges; on the right, quality issues, both internal and external. [3]

16

维度2： 技术债务的有意性

- ◆ 有意技术债务
 - 主动承担的债务（我们有意承诺解决）
 - 例如：“若本次发布未能完成，将不会有第二次发布。让我们专注于编码，尽量减少测试工作量。不编写文档。发布后几天内直接让用户操作系统，而非提前进行培训。”
 - 例如：“我们没有时间构建多平台通用支持。当前只支持iOS，Android等平台后续再开发。代码只需在iOS应用中运行良好即可。”
 - 例如：“尽管我们采用测试驱动开发方法，但项目最后两个月编写的代码仍来不及为所有代码编写单元测试。我们将在软件发布后再编写这些测试。”

维度2： 技术债务的有意性

- ◆ 非故意技术债务
 - 因低质量工作而无意中产生的债务。
 - 例如：初级程序员编写了低质量代码
 - 例如：新人/承包商编写的代码未遵循编码规范
 - 例如：重大设计策略（如决定采用模型-视图-控制器架构连接软件所有核心模块）效果不佳
 - 例如：全面重构任务出现偏差
 - 例如，无心失失。“真希望我们早知道框架2.1会比框架2.0好这么多。”
 - 例如：粗心失误。“设计？什么设计？硬件设计？”

19

维度3：

时间跨度

- ◆ 短期债务
 - 通常被动产生，且行动在短期内完成
 - 例如：为赶发布进度而跳过部分集成测试
 - 例如："没时间按规范实现，先硬塞进去，发布后再修复。"
 - 例如：为紧急部署关键补丁而违反编码规范
- ◆ 长期债务
 - 通常出于战略考虑而主动承担
 - 例如："我们认为至少三年内无需支持第二平台，因此设计仅支持单一平台。"
 - 例如：核心部分采用COBOL编写已达40年。该模块应继续采用COBOL作为编程语言。

20

维度4：

关注程度

- ◆ 短期债务集中度
 - 可单独识别的捷径（例如类的部分实现）
- ◆ 非重点短期债务
 - 大量细微捷径（例如频繁违反编码规范或文档松散）
- ◆ 聚焦型长期债务
 - 例如："我们认为至少三年内无需支持第二个平台，因此我们的设计仅支持单一平台。"
- ◆ 未聚焦的长期债务
 - 例如：让我们采用最高效的语言和框架来实现各个功能模块。

21

识别技术债务

22

增加技术债务的模式

- ◆ 软件开发中存在一些众所周知的*模式会增加技术债务*
 1. 时间压力
 2. 代码重复
 3. 追求"一次到位"

23

增加技术债务的模式：

1. 时间压力

- ◆ 当团队被迫履行不合理的承诺时，必然会偷工减料以满足管理层的期望。
 - 例如：新功能范围不断扩大、团队构成变更、集成延迟
- ◆ 解决方案
 - 采用更灵活的规划方法

增加技术债务的模式：

2. 代码重复

- ◆ 代码重复的诸多原因
 - 团队成员经验不足
 - 复制粘贴的编程习惯
 - 遵循现有软件的劣质设计
 - 交付压力；猜测进度安排
- ◆ 解决方案
 - 使用静态分析工具辅助（参见《代码质量》幻灯片）
 - 结对编程
 - 传播知识并提升团队成员能力
 - (a) 立即偿还技术债务，或 (b) 记录待后续处理
 - (1) 立即修复；(2) 将该技术债务添加至运行时异常列表，稍后处理；(3) 将债务信息（位置、描述、不修复的潜在成本）加入项目待办事项列表

增加技术债务的模式：

3. 首次就做到"正确"

- ◆ 避免重复劳动——前期构建大量强大的设计方案。
 - 我们过度设计并过度泛化了目标产品。
- ◆ 修复成本持续攀升

管理技术债务

管理技术债务 [5]

- ◆ 若无法避免技术债务，就必须进行管理。
- ◆ 这意味着要识别技术债务、追踪其动态、做出合理决策，并防止其最严重的后果。

管理技术债务的方法：
追踪技术债务 [2]

- ◆ 追踪技术债务（TD）是管理它的必要手段。
 - 所有“良性债务”均可追踪（至少按定义而言）
 - 记录为缺陷
 - 纳入产品待办事项列表
 - 监控项目速度
 - 监控返工量
 - ◆ 衡量债务的方法
 - 产品待办事项列表中的总技术债务
 - 维护预算（或维护预算占比）
- 是无生产力的维护工作

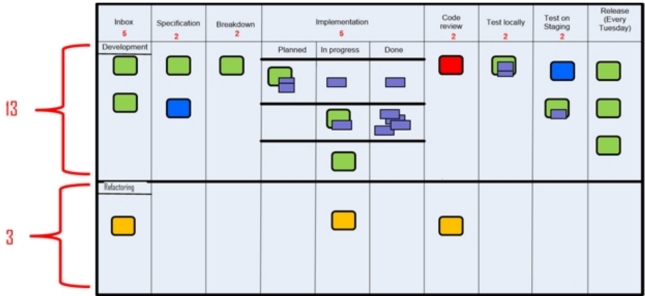
管理技术债务的方法：

- 以金钱而非功能来衡量技术债务，例如“研发预算的50%”

是无生产力的维护工作

管理技术债务的方法：
可视化追踪技术债务 [6]

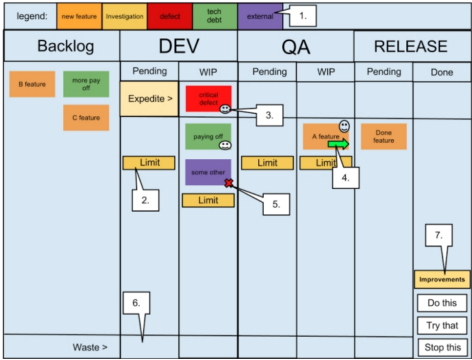
- ◆ 在任务看板上可视化技术债务。下部区域展示项目中待处理的三项重构任务（即技术债务实例）：其中一项正在代码审查中，另一项处于“进行中”状态，最后一项尚未着手处理。



管理技术债务的方法：

可视化追踪技术债务 [6]

◆ 在看板上可视化技术债务。（1-7项请参见下一张



31

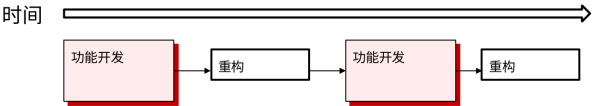
上一页幻灯片中的七个图例说明

- ◆ 图例说明——此处展示看板上可放置的卡片类型（按颜色区分），包括：新功能、技术债务、缺陷、调查事项（支持类）及外部问题（如新增服务器）。此举旨在清晰可视化当前处理的工作类型
- ◆ 限制——每个待办事项和进行中事项列都设有上限，用于控制可承接的工作量。这些限制基于资源水平及其他瓶颈设定，旨在鼓励在存在余力时主动提取工作，而非持续向瓶颈环节强行塞入更多任务。
- ◆ 化身——每位团队成员拥有一个专属化身，用于标记其负责的任务项。此举旨在告知其他成员该卡片正在处理中且应持续推进。鉴于我们多数开发工作采用结对模式，每个开发任务上通常会出现多个化身。
- ◆ 箭头——标示任务自上次站立会议后的进展方向，此亦为流程可视化的一部分。这些卡片在每日站会中常被忽略，因为我们需要聚焦于进展停滞的项目，例如……
- ◆ 阻碍项——我们需最大限度减少阻碍项，因其会加剧系统瓶颈，故用“阻塞”头像明确标注此类事项。阻塞卡片时还需补充说明“谁”导致阻塞及“为何”无法推进。
- ◆ 浪费项——任务可在流程任一阶段终止，我们将收集这些废弃项并在每周回顾会议上进行评估。
- ◆ 持续改进——此外，在每周回顾会上，我们会确定三项希望在下周推进的改进事项。这些事项会被汇总在看板上，并在每日晨会开始时重新确认。若某项改进未能取得进展，则可能顺延至下周处理。

管理技术债务的方法：

偿还技术债务

◆ 有效策略是定期预留时段处理技术债务。



- ◆ 更长的重构时间可换取更少的重构周期，反之亦然。
 - 优先处理重构任务，以先解决关键领域。
 - 让产品负责人和利益相关者参与进来，获取他们的观点

33

管理技术债务的方法：

讨论技术债务

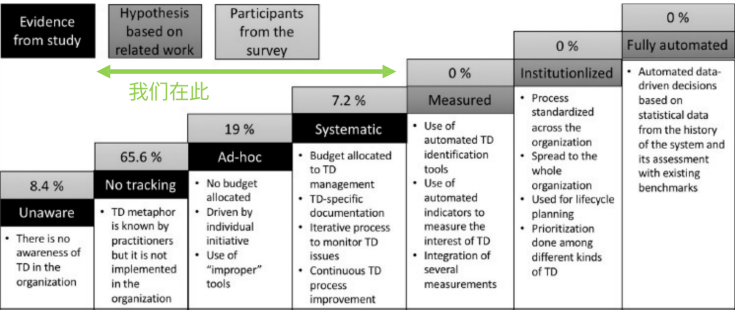
- ◆ 向技术人员普及项目涉及的业务决策知识
 - 将开发人员融入业务背景
- ◆ 向业务人员传授技术决策知识
 - 同时作为其业务或运营决策的输入要素之一
- ◆ 提升对重要问题的认知度/透明度（这些问题常被掩盖）
 - 作为规避风险的措施
- ◆ 使技术债务管理更具可视化与明确性
 - 作为逐项解决问题的项目起点。

34

实践中的技术债务管理

- ◆ 尽管开发人员可能渴望采用更优方法处理这些事，却未能付诸实践。
- ◆ 职场策略（基于对26家企业的调查[5]）
 1. 无所作为——"没坏就别修"——因为技术债务可能永远不会暴露给客户。
 2. 采用风险管理方法评估技术债务的成本与价值并设定优先级。例如：将每个发布周期的5%至10%资源用于处理技术债务。
 3. 通过让客户和非技术利益相关者成为平等合作伙伴，并促进关于技术债务影响的开放对话，来管理他们的期望。
 4. 与开发团队共同开展技术审计，使技术决策透明化并明确化；通过待办事项/任务看板进行追踪。

实践中的技术债务管理 [7]

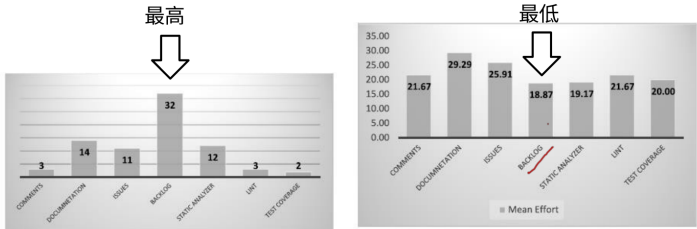


具有不同角色的组织（开发人员、架构师、管理者）

实践中的技术债务管理 [7]

调查背景[7]：来自15家大型企业的226名参与者

实践中的技术债务管理 [7]



具有不同角色的组织（开发人员、架构师、管理者）

实践中的技术债务管理 [7]

- ◆ [有效] 待办事项列表、静态分析工具和“lint”程序都能提升追踪水平，但我们尚未观察到显著差异（尽管静态代码分析工具似乎更能提升参与者的意识）。这些工具的开销也最小，因此目前被视为追踪技术债务的最佳实践。
 - [有效] 待办事项列表是参与者使用最广泛的工具。其中最常用的待办事项工具是Jira、Hansoft和Excel。
- ◆ [无效] 代码注释有助于提升认知度，但不被视为追踪手段，仅1%受访者采用。这可能是因为注释未应用于可被团队监控的代码外部文档。
- ◆ [无效] 文档记录能提升TD意识，但不被视为高级追踪手段，且开销最高。主要工具为Microsoft Excel或Word。相较于其他方法，该做法显然不值得推荐。
- ◆ [无效] 相较于其他方法，使用缺陷管理系统追踪TD并未显著提升认知度或追踪效果，且其开销略高。由此可推断此法亦非追踪TD的最佳方案。
- ◆ [无效] 测试覆盖率虽未产生显著开销，但对认知度和追踪水平的贡献有限。这可能源于测试覆盖率仅涉及测试覆盖率的极小部分。

参考文献与资源

1. Ward Cunningham. 1992. 《WyCash投资组合管理系统》。载于《面向对象编程系统、语言及应用会议论文集增刊》（OOPSLA '92），Jerry L. Archibald与Mark C. Wilkes编。ACM出版社，纽约，美国，第29-30页。DOI-<http://dx.doi.org/10.1145/157709.157715>
2. S. McConnell, “技术债务管理（幻灯片）”, 技术债务管理研讨会（ICSE 2013 组成部分）： IEEE, 2013
 - <http://2013.icse-conferences.org/documents/publicity/MTD-WS-McConnell-slides.pdf>
3. Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. 2012. 《技术债务：从隐喻到理论与实践》。IEEE软件杂志，第29卷第6期（2012年11月），第18-21页。DOI: <https://doi.org/10.1109/MS.2012.167>
4. 克里斯·斯特林，2010. 《管理软件债务：为不可避免的变化而构建》。艾迪生-韦斯利专业出版社；第1版，ISBN-10：0321948610. [了解技术债务的优质资源]
5. 林、N. 塔克桑德与C. 西曼，《平衡之道：软件从业者对技术债务的见解》，IEEE软件期刊。DOI: 10.1109/MS.2012.130
6. Jesper Boeg, 成功平衡技术债务与新功能——保持在“流暢区”或如何重返该状态。 <http://agileupgrade.com/successfully-balancing-technical-debt-and-new-features-staying-in-the-flow-zone-or-how-to-get-back-in-there/>
7. 安东尼奥·马蒂尼、特塞斯·贝斯克、扬·博施：技术债务追踪：实践现状调查及15家大型组织的案例研究。《科学计算与程序》163: 42-61 (2018)
8. Terese Besker, Antonio Martini 与 Jan Bosch. 2019. 《技术债务在待办事项管理中的分级处理》。载于《第二届国际技术债务会议论文集（TechDebt '19）》。IEEE出版社，美国新泽西州皮斯卡塔韦，第13-22页。DOI: <https://doi.org/10.1109/TechDebt.2019.00010>